

Here is a simple mentor-explanation script you can speak almost directly.

Project Explanation Script

Good morning sir/ma'am.

I will explain the structure of this ERP React project in a simple way.

This project is a frontend ERP application built using React, TypeScript, and Vite. ERP means Enterprise Resource Planning. So this app has different business modules like Sales, Inventory, Warehouse, Procurement, Finance, HR, Documents, Maintenance, and Supply Chain.

The project is mainly divided into UI screens, data handling files, reusable React hooks, styling files, and helper utility files.

The app starts from `index.html`. This file has one root div where React is inserted.

After that, React starts from `src/main.tsx`. This file loads the main `App.tsx` component and also imports the main CSS file.

The most important file is `src/app/App.tsx`. This file controls the main layout of the app. It has the login page, sidebar, header, dashboard, navigation, and many ERP screens. When the user clicks a module in the sidebar, `App.tsx` decides which screen should be shown.

For example, if the user clicks Warehouse, React shows the Warehouse module. If the user clicks Inventory, React shows the Inventory module.

Now I will explain the folders.

First, the `src` folder is the main source code folder. Almost all React code is inside this folder.

Inside `src`, there is an `app` folder. This contains the main application file, mainly `App.tsx`. This file is like the controller of the whole frontend. It decides what the user sees after login.

Inside `src/app/components/ui`, there are reusable UI components like buttons, inputs, dialogs, tables, tabs, cards, badges, and sidebars. These are common design parts. Instead of creating a new button or table again and again, the app reuses these files.

Then we have the `src/components` folder. This folder contains the main ERP module screens. For example, there are files like `SalesModuleComplete.tsx`, `InventoryModuleComplete.tsx`, `WarehouseModuleComplete.tsx`, `ProcurementModuleComplete.tsx`, `DocumentsModuleComplete.tsx`, and others.

Each of these files represents one big ERP page. For example, `InventoryModuleComplete.tsx` shows inventory items, stock details, search, add, edit, delete, and export options.

Next is the `src/hooks` folder. This folder contains reusable React helper functions. The main files are `useERP.ts` and `useWarehouse.ts`.

These hooks help the UI screens work with data. For example, a screen does not directly write all the add, edit, delete logic. Instead, it uses hooks like `useInventory`, `usePurchaseOrders`, or `useReceiving`.

So the hook acts like a bridge between the screen and the data service.

Next is the `src/services` folder. This is one of the most important folders. It handles the data of the app.

There are three main files here:

`erpDataServices.ts` handles most ERP data like sales, inventory, finance, HR, payroll, projects, purchase orders, employees, invoices, and so on.

`warehouseService.ts` handles warehouse-specific data like receiving, picking, packing, dispatch, dock, zones, barcodes, shipment tracking, and GPS.

`recordMetaService.ts` handles comments, attachments, and timeline history for records.

Currently, these service files store data in browser `localStorage`. That means this project does not yet use a real backend database. The data is saved inside the browser.

Later, if we want to connect this project to Supabase, Firebase, or any backend, the `services` folder is the best place to change. The UI can remain mostly the same, and the service files can be updated to talk to the database.

Next is the `src/utils` folder. This folder has helper functions that are not directly UI and not directly data storage.

For example, `erpExportUtils.ts` and `exportUtils.ts` help export data to CSV, Excel-like files, PDF-like reports, or print format.

If the user clicks Export in Inventory or Sales, the screen calls a function from the utils folder.

Next is the `src/styles` folder. This folder contains CSS files. It controls the look of the app, like colors, fonts, spacing, light mode, dark mode, and theme values.

The main style file is `index.css`, and it imports other style files like `fonts.css`, `tailwind.css`, and `theme.css`.

There is also an `imports/pasted_text` folder. This contains markdown files with design notes, feature lists, and project references. These files are not directly used by React while running the app, but they help developers understand the intended design and features.

Outside `src`, there are some project-level files.

`package.json` tells which libraries are used and what commands can be run. For example, `npm run dev` starts the project and `npm run build` builds it.

`vite.config.ts` contains Vite setup. Vite is the tool used to run and build this React project.

`tsconfig.json` contains TypeScript settings.

`node_modules` contains installed packages like React, Vite, icons, charts, and UI libraries.

`dist` contains the final built version of the app after running the build command.

Now I will explain the data flow.

Suppose the user adds a new inventory item.

First, the user fills a form in the Inventory screen. That screen is inside `src/components`.

Then the screen calls a hook from `src/hooks`, for example `useInventory`.

That hook calls a service from `src/services`, for example the inventory service inside `erpDataServices.ts`.

The service saves the data in `localStorage`.

After saving, React updates the screen and the new item appears in the table.

So the simple flow is:

```
User clicks button
-> React screen handles the click
-> Hook manages the action
-> Service saves or reads data
-> React updates the screen
```

Another example is export.

If the user clicks Export in the Sales module:

```
Sales screen
-> calls export function from utils
-> browser downloads CSV or report
```

So the project is organized like this:

```
UI screens are in components and App.tsx
Reusable UI parts are in app/components/ui
Data helper logic is in hooks
Actual data saving logic is in services
Export and common tools are in utils
Styling is in styles
```

The most important files to understand first are:

```
src/main.tsx
src/app/App.tsx
src/components/InventoryModuleComplete.tsx
src/components/WarehouseModuleComplete.tsx
src/hooks/useERP.ts
src/hooks/useWarehouse.ts
src/services/erpDataServices.ts
src/services/warehouseService.ts
```

```
src/utils/erpExportUtils.ts
src/styles/index.css
```

If I understand these files, I can understand most of the project.

In short, this project is a React-based ERP frontend. It already has many ERP modules and screens. It uses localStorage for temporary data storage. The structure is good for a frontend demo, and later the services folder can be connected to a real backend like Supabase.

One-Minute Short Version

This is a React ERP frontend project. The app starts from `index.html`, then `src/main.tsx`, then `src/app/App.tsx`.

`App.tsx` controls login, layout, sidebar, header, and which ERP module is shown.

The `src/components` folder contains big ERP screens like Sales, Inventory, Warehouse, Procurement, Documents, Maintenance, and Supply Chain.

The `src/hooks` folder contains reusable React logic for add, edit, delete, search, filter, sort, and pagination.

The `src/services` folder handles data. Right now it saves data in browser localStorage. Later this folder should connect to Supabase or another backend.

The `src/utils` folder contains helper functions like export to CSV, Excel, PDF, and print.

The `src/styles` folder controls the visual design.

So the basic flow is:

```
Screen -> Hook -> Service -> localStorage -> Screen updates
```

Simple Table For Mentor

Folder/File	Purpose	Easy Example
<code>index.html</code>	HTML page where React is placed	Contains root div
<code>src/main.tsx</code>	Starts the React app	Loads <code>App.tsx</code>
<code>src/app/App.tsx</code>	Main app control file	Shows Dashboard or Inventory
<code>src/app/components/ui</code>	Reusable UI parts	Button, table, dialog
<code>src/components</code>	ERP module screens	Sales, Warehouse, Inventory
<code>src/hooks</code>	Reusable React logic	<code>useInventory()</code>
<code>src/services</code>	Data saving and reading	Save purchase order
<code>src/utils</code>	Helper tools	Export table to CSV

Folder/File	Purpose	Easy Example
src/styles	CSS and theme	Dark mode colors
src/imports	Reference notes	Feature/design markdown
package.json	Project packages and commands	npm run dev
vite.config.ts	Vite setup	React + Tailwind setup
node_modules	Installed libraries	React, icons, charts
dist	Final build output	Deployment files